# SHOPIFY

*DSA-Endsem project*

SEC-4

# TEAM-4

- T Varsha - 2510030170
- K Laasya -2510030368
- Shloka-2510030025
- Shrenika-2510030022
- Anvitha-2510030026
- M Hasini -2510030098
- K Meghana-2510030099
- B Charitha-2510040026

# ABSTRACT

This project is a console-based shopping system developed using Java. It allows users to view products, search, sort, and manage a shopping cart efficiently. Various data structures like ArrayList, LinkedList, Stack, Queue, and HashMap are used to implement features such as searching, sorting, undo/redo operations, and order processing. The system also includes discount calculation and file handling for product data. This project demonstrates practical implementation of data structures in a real-world application.

03

# MODULE

- Product Module
  - Implemented using Class & Objects (OOP)
  - Stores product details (id, name, price, stock)
- File Handling Module
  - Uses BufferedReader + FileReader
  - Reads product data from file into ArrayList
- Searching Module
  - Linear Search → Search by Product ID
  - Binary Search → Search by Product Name
- Sorting Module
  - Merge Sort → Sort products by Price
  - Quick Sort → Sort products by Name
  - Comparator → Sort by ID
- Cart Module
  - LinkedList → Store cart items
  - Stack → Undo/Redo operations
  - Handles add, remove, total calculation
- Hashing Module
  - Uses HashMap
  - Fast product access using ID as key
- Discount Module
  - Implemented using Conditional Statements (if-else)
  - Applies discount based on total amount
- Order Processing Module
  - Uses Queue (FIFO)
  - Handles order placement and processing
- Main Application Module
  - Uses Menu-driven program (Scanner + Switch Case)
  - Integrates all modules together

# CORE MODULES

04

- **Product Module**

The Product module defines the structure of each item using object-oriented programming concepts. It stores key attributes such as product ID, name, price, and stock. It also includes a method to display product details in a user-friendly format. This module acts as the foundation for all other operations.

- **File Handling Module**

The File Handling module is responsible for reading product data from an external file. It uses BufferedReader and FileReader to efficiently load data. The data is then stored in an ArrayList for easy access and manipulation. This ensures dynamic and scalable data management.

- **Searching Module**

The Searching module enables efficient product lookup within the system. It uses Linear Search to find products by ID and Binary Search to locate products by name. Binary search improves performance by working on sorted data. This module ensures quick and accurate search results.

# PROCSSING MODULE

- **Sorting Module**

The Sorting module organizes products based on different criteria for better usability. It uses Merge Sort to sort products by price and Quick Sort to sort by name. Additionally, Comparator is used for sorting by ID. This improves data readability and user experience.

- **Cart Module**

The Cart module manages all shopping cart operations within the system. It uses a LinkedList to store cart items dynamically. Stack data structures are used to implement undo and redo functionalities. It also handles adding, removing, and calculating the total cost of items.

- **Hashing Module**

The Hashing module improves performance by enabling fast data retrieval. This allows instant access to products without searching through the entire list. It significantly reduces time complexity for lookup operations.

# CONTROL MODULES

- **Discount Module**

The Discount module applies price reductions based on the total purchase amount. It uses conditional statements to determine the discount percentage. Higher purchase amounts receive greater discounts. This enhances the shopping experience and encourages bulk purchases.

- **Order Module**

The Order module handles order placement and processing efficiently. It uses a Queue data structure to follow the First-In-First-Out (FIFO) principle. This ensures that orders are processed in the correct sequence. It maintains a smooth and organized workflow.

- **Main Application Module**

The Main Application module acts as the control center of the system. It provides a menu-driven interface using switch-case statements. It integrates all modules and manages user interactions. This module ensures the smooth functioning of the entire application.

Product.java | FileHandler.... | Search.java | Cart.ja

```java
    public void add(Product p) {

        if (p == null) {
            System.out.println("Invalid product");
            return;
        }

        if (p.stock > 0) {
            cart.add(p);
            undo.push(p);
            p.stock--;
            System.out.println("Added: " + p.name);
        } else {
            System.out.println("Out of stock!");
        }
    }

    public void remove(Product p) {
        if (cart.remove(p)) {
            p.stock++;
            System.out.println("Removed");
        }
    }

    public void undo() {
        if (!undo.isEmpty()) {
            Product p = undo.pop();
            cart.remove(p);
            redo.push(p);
            p.stock++;
            System.out.println("Undo done");
        }
    }

    public void redo() {
        if (!redo.isEmpty()) {
            add(redo.pop());
        }
    }
```

Product.java | FileHandler.... | Search.java | Cart.java | OrderQ

```java
package search;

import java.util.*;

public class Search {

    public static Product linearById(ArrayList<Product> list, int id) {
        for (Product p : list)
            if (p.id == id) return p;
        return null;
    }

    public static Product binaryByName(ArrayList<Product> list, String key) {
        int l = 0, r = list.size() - 1;

        while (l <= r) {
            int mid = (l + r) / 2;

            int cmp = list.get(mid).name.compareToIgnoreCase(key);

            if (cmp == 0) return list.get(mid);
            else if (cmp < 0) l = mid + 1;
            else r = mid - 1;
        }
        return null;
    }
}
```

```java
//  QUICK SORT → Ascending by Price
public void sortByAscending(Product arr[], int low, int high) {

    if (low < high) {
        int pi = partition(arr, low, high);

        sortByAscending(arr, low, pi - 1);
        sortByAscending(arr, pi + 1, high);
    }
}

int partition(Product arr[], int low, int high) {

    double pivot = arr[high].price;
    int i = low - 1;

    for (int j = low; j < high; j++) {

        if (arr[j].price < pivot) {
            i++;
            swap(arr, i, j);
        }
    }

    swap(arr, i + 1, high);
    return i + 1;
}

void swap(Product arr[], int i, int j) {

    Product temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
```

# CODE

```java
//  MERGE SORT → Descending by Price
public void sortByDescending(Product arr[], int low, int high) {

    if (low < high) {
        int mid = (low + high) / 2;

        sortByDescending(arr, low, mid);
        sortByDescending(arr, mid + 1, high);

        merge(arr, low, mid, high);
    }
}

void merge(Product arr[], int low, int mid, int high) {

    int i = low, j = mid + 1, k = 0;
    Product temp[] = new Product[high - low + 1];

    while (i <= mid && j <= high) {

        if (arr[i].price > arr[j].price)
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid) temp[k++] = arr[i++];
    while (j <= high) temp[k++] = arr[j++];

    for (int x = low, y = 0; x <= high; x++, y++)
        arr[x] = temp[y];
}
```

```java
//  SELECTION SORT → By Name
public void sortByName(Product arr[]) {

    for (int i = 0; i < arr.length - 1; i++) {

        int minIndex = i;

        for (int j = i + 1; j < arr.length; j++) {

            if (arr[j].name.compareToIgnoreCase(arr[minIndex].name) < 0) {
                minIndex = j;
            }
        }

        //  correct swap (outside loop)
        Product temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}
```

```java
package order;

import java.util.*;

public class OrderQueue {

    Queue<Product> q = new LinkedList<>();

    public void placeOrder(List<Product> list) {
        q.addAll(list);
    }

    public void process() {

        double total = 0;

        System.out.println("\n--- BILL (FIFO) ---");

        while (!q.isEmpty()) {
            Product p = q.poll();
            System.out.println(p.name + " - " + p.price);
            total += p.price;
        }

        System.out.println("Total: " + total);
    }
}
```

# CODE

```
Console
MainApp [Java Application] C:\Users\Admin\.p2\pool\plugins\c

========= WELCOME TO SHOPIFY =========
1. View All Products
2. Search Products (Name / ID)
3. Sort Products
4. Add Item to Cart
5. Remove Item from Cart
6. Undo Last Action
7. Redo Last Action
8. View cart
9. Place Order & Generate Bill
10.Exit
=========================================
Enter your choice: 1

Available Products:
101 | Shoes | 8999.0 | Stock:20
202 | Laptop | 95499.0 | Stock:15
303 | Jeans | 3999.0 | Stock:45
404 | Mobile | 67999.0 | Stock:50
505 | Watch | 6499.0 | Stock:72
606 | Bag | 2499.0 | Stock:88
707 | Camera | 25000.0 | Stock:21
808 | Speaker | 4000.0 | Stock:30
909 | Tablet | 20000.0 | Stock:22
1011 | Earbuds | 2500.0 | Stock:49
```

```
Console
MainApp [Java Application] C:\Users\Admin\.p2\pool\plugins

========= WELCOME TO SHOPIFY =========
1. View All Products
2. Search Products (Name / ID)
3. Sort Products
4. Add Item to Cart
5. Remove Item from Cart
6. Undo Last Action
7. Redo Last Action
8. View cart
9. Place Order & Generate Bill
10.Exit
=========================================
Enter your choice: 2

Search Options:
1. Search by Name (Binary Search)
2. Search by ID (Linear Search)
Choose option: 1
Enter product name: bag
Found: 606 | Bag | 2499.0 | Stock:88
```

```
========= WELCOME TO SHOPIFY =========
1. View All Products
2. Search Products (Name / ID)
3. Sort Products
4. Add Item to Cart
5. Remove Item from Cart
6. Undo Last Action
7. Redo Last Action
8. View cart
9. Place Order & Generate Bill
10.Exit
=========================================
Enter your choice: 4
Enter product ID: 202
Enter quantity: 4
Added: Laptop
Added: Laptop
Added: Laptop
Added: Laptop
4 item(s) added to cart
```

# OUTPUT

```
========= WELCOME TO SHOPIFY =========
1. View All Products
2. Search Products (Name / ID)
3. Sort Products
4. Add Item to Cart
5. Remove Item from Cart
6. Undo Last Action
7. Redo Last Action
8. View cart
9. Place Order & Generate Bill
10.Exit
=========================================
Enter your choice: 6
Undo done

=====ᵉ=== WELCOME TO SHOPIFY ᵉ========ᵉ
1. View All Products
2. Search Products (Name / ID)
3. Sort Products
4. Add Item to Cart
5. Remove Item from Cart
6. Undo Last Action
7. Redo Last Action
8. View cart
9. Place Order & Generate Bill
10.Exit
=====ᵉ=======ᵉ=======ᵉ=======ᵉ=======ᵉ====
Enter your choice: 8

Your Cart:
202 | Laptop | 95499.0 | Stock:13
202 | Laptop | 95499.0 | Stock:13
Total: 190998.0
```

```
========= WELCOME TO SHOPIFY =========
1. View All Products
2. Search Products (Name / ID)
3. Sort Products
4. Add Item to Cart
5. Remove Item from Cart
6. Undo Last Action
7. Redo Last Action
8. View cart
9. Place Order & Generate Bill
10.Exit
=========================================
Enter your choice: 6
Undo done

=====ᵉ=== WELCOME TO SHOPIFY ᵉ========ᵉ
1. View All Products
2. Search Products (Name / ID)
3. Sort Products
4. Add Item to Cart
5. Remove Item from Cart
6. Undo Last Action
7. Redo Last Action
8. View cart
9. Place Order & Generate Bill
10.Exit
=====ᵉ=======ᵉ=======ᵉ=======ᵉ=======ᵉ====
Enter your choice: 8

Your Cart:
202 | Laptop | 95499.0 | Stock:13
202 | Laptop | 95499.0 | Stock:13
Total: 190998.0
```

```
202 | Laptop | 95499.0 | Stock:4
202 | Laptop | 95499.0 | Stock:4
202 | Laptop | 95499.0 | Stock:4
Total Amount: 1050489.0

Confirm Order? (yes/no): yes

Select Payment Method:
1. UPI
2. Card
3. Cash on Delivery
1
Processing UPI Payment...
Payment Successful ☑
50% Discount Applied
Order placed in queue
Processing Order...
202 | Laptop | 95499.0 | Stock:4
202 | Laptop | 95499.0 | Stock:4
202 | Laptop | 95499.0 | Stock:4
202 | Laptop | 95499.0 | Stock:4
202 | Laptop | 95499.0 | Stock:4
202 | Laptop | 95499.0 | Stock:4
202 | Laptop | 95499.0 | Stock:4
202 | Laptop | 95499.0 | Stock:4
202 | Laptop | 95499.0 | Stock:4
202 | Laptop | 95499.0 | Stock:4
202 | Laptop | 95499.0 | Stock:4
Order Completed ☑
Final Bill after Discount: 525244.5
```

# OUTPUT

# LIST OF TOPICS USED

**Data Structures**: ArrayList, LinkedList, Stack, Queue
**Algorithms**: Linear Search, Binary Search, Merge Sort, Quick Sort
**Concepts**: OOP, File Handling, Comparator, Conditional Statements, Loops, Switch-Case

# FUTURE ENHANCEMENT

1. GUI-Based Application
   - Convert the console app into a graphical user interface (GUI) using:
     - Java Swing / JavaFX
2. Database Integration
   - Replace in-memory product storage with a real database like:
     - MySQL / MongoDB
3. User Authentication System
   - Add login/signup functionality:
     - customer (shop items)
4. Online Payment Integration
   - Integrate payment gateways like:
     - UPI / Credit Card / Net Banking
   - Enables complete e-commerce functionality.
5. Product Recommendation System
   - Suggest products based on:
     - Previous purchases
     - Popular items
   - Uses basic ML or simple logic.
6. Performance Optimization
   - Improve efficiency by:
   - Using better data structures (HashMap, TreeSet)
   - Optimizing sorting/search algorithms

# WHAT WE HAVE LEARNT

OUTCOME / WHAT WE LEARNED

- GAINED UNDERSTANDING OF OBJECT-ORIENTED PROGRAMMING (OOP) CONCEPTS LIKE CLASSES, OBJECTS, ENCAPSULATION, AND MODULAR DESIGN.
- IMPLEMENTED AND COMPARED SEARCHING ALGORITHMS (LINEAR SEARCH, BINARY SEARCH).
- APPLIED SORTING TECHNIQUES SUCH AS QUICK SORT AND MERGE SORT.
- LEARNED THE USE OF DATA STRUCTURES LIKE ARRAYLIST, STACK (UNDO/REDO), AND QUEUE (ORDER PROCESSING).
- UNDERSTOOD REAL-TIME APPLICATION OF DSA CONCEPTS IN E-COMMERCE SYSTEMS.
- DEVELOPED LOGIC FOR CART MANAGEMENT AND BILLING SYSTEM.
- IMPROVED PROBLEM-SOLVING AND ALGORITHMIC THINKING SKILLS.
- LEARNED HOW TO DESIGN A MENU-DRIVEN CONSOLE APPLICATION.

# THANK YOU